

Git Cheat Sheet

Everyday commands, with the reasons and use cases behind them

cicd-demo learning notes

September 2026

1. Reading Git's help and syntax

Syntax notation (works for most CLI tools)

Symbol	Meaning
[x]	Optional: you can leave it out
<x>	A value you fill in (branch name, file, commit)
--	End of options; what follows is paths/arguments
...	Can be repeated
(a b)	Pick one of them

Example: `git branch (-m | -M) [<old-branch>] <new-branch>`

`old-branch` is optional (defaults to the **current branch**);
`new-branch` is required.

Getting help

Command	You get
<code>git log -h</code>	Short usage summary in the terminal
<code>git log --help</code>	Full manual (opens in the browser on Windows)
<code>git help log</code>	Same as <code>--help</code>

Why is `git log -h` so short? Most of its options (`--graph` , `--oneline` , `--all`) come from shared parts of Git, so `-h` doesn't list them. Use `--help` and press Ctrl+F. The same usage text also shows `git show` , because both commands share one source file (`builtin/log.c`).

2. One-time setup and starting a repo

Command	What it does / when to use it
<code>git config --global user.name "Your Name"</code>	Name recorded in every commit. Set once per machine.
<code>git config --global user.email "you@mail.com"</code>	Email recorded in commits (use the same one as GitHub).
<code>git config --global init.defaultBranch main</code>	New repos start on <code>main</code> instead of <code>master</code> .
<code>git config --global alias.lg "log --graph --oneline --all"</code>	Creates a shortcut, so <code>git lg</code> shows the branch graph.
<code>git config --list</code>	Show all current settings.
<code>git init</code>	Turn the current folder into a Git repository.
<code>git clone <url></code>	Download a repository from GitHub (with full history).
<code>git remote add origin <url></code>	Connect a local repo to GitHub. <code>origin</code> is the usual name.
<code>git remote -v</code>	Show which remote URLs are connected.

First push of a new project (the commands GitHub shows you):

```
git init
git add .
git commit -m "first commit"
git branch -M main          # rename current branch to main (see section 4)
git remote add origin https://github.com/you/repo.git
git push -u origin main     # -u = remember origin/main as upstream
```

3. The daily save-and-share loop

Command	What it does / when to use it
<code>git status</code>	What changed, what's staged, which branch you're on. Run it often.
<code>git diff</code>	Changes not yet staged (line by line).
<code>git diff --staged</code>	Changes that will go into the next commit.
<code>git add <file></code>	Stage one file for the next commit.
<code>git add .</code>	Stage everything in the current folder (check <code>git status</code> first).
<code>git commit -m "message"</code>	Save staged changes as a commit. Write <i>what and why</i> .
<code>git fetch</code>	Download new commits from GitHub without changing your files.
<code>git pull</code>	<code>fetch</code> + merge into your current branch.
<code>git push</code>	Upload your commits to the branch's upstream on GitHub.
<code>git push -u origin <branch></code>	First push of a new branch; sets its upstream.

The three areas: Working folder → `git add` → Staging area → `git commit` → Local repo → `git push` → GitHub.

4. Branches

Listing branches

Command	Shows
<code>git branch</code>	Local branches; * marks the current one
<code>git branch -l</code>	Same as above (<code>-l</code> = <code>--list</code>)
<code>git branch -l "fix/*"</code>	Only branches matching the pattern
<code>git branch -r</code>	Remote branches (e.g. <code>origin/main</code>)
<code>git branch -a</code>	Local + remote
<code>git branch -v</code>	Local branches + last commit on each
<code>git branch -vv</code>	Also shows each branch's upstream , e.g. <code>[origin/main]</code>

`git branch` vs `git branch -l`: with no name they are identical. With a name they differ:

`git branch feature*` **creates** a branch literally called `feature*`.

`git branch -l "feature*"` **lists** branches starting with `feature`.

(Before Git 2.20, `-l` meant "create reflog", so old tutorials may use it differently.)

Creating and switching

Command	What it does / when to use it
<code>git switch <branch></code>	Move to an existing branch.
<code>git switch -c <branch></code>	Create a new branch and move onto it . The everyday way to start new work (same as the older <code>git checkout -b</code>).
<code>git switch -</code>	Go back to the previous branch.
<code>git branch <branch></code>	Create a fresh branch but stay where you are.
<code>git branch -c <new></code>	Copy the current branch to a new name, including its settings (upstream) and reflog . You stay on the current branch.

`git switch -c` vs `git branch -c`

	<code>git switch -c name</code>	<code>git branch -c name</code>
<code>-c</code> means	create	copy
New branch at current commit	Yes	Yes
Switches you to it	Yes	No, you stay on the current branch
Copies upstream/config + reflog	No (fresh branch)	Yes
Daily use	Very common	Rare

Pitfall: `git branch -c main1` then pushing.

`git push origin main1` creates a **new** `main1` branch on GitHub. It does **not** touch GitHub's `main`.

But `main1` **copied main's upstream** (`origin/main`). So on `main1`, `git pull` pulls from GitHub's **main**. A plain `git push` fails with the default settings, and with `push.default=upstream` it would push into **main**.

Fix: `git push -u origin main1` (sets its own upstream), then check with `git branch -vv`.

Better: create it with `git switch -c main1`, which starts with no copied upstream.

Renaming, copying, deleting

Command	What it does
<code>git branch -m <new></code>	Rename the current branch. Fails if <new> already exists (safe).
<code>git branch -m <old> <new></code>	Rename a specific branch.
<code>git branch -M <new></code>	Force rename: overwrites an existing branch with that name (= <code>-m --force</code>).
<code>git branch -C <new></code>	Force copy (overwrites an existing branch).
<code>git branch -d <branch></code>	Delete a branch that is already merged (safe).
<code>git branch -D <branch></code>	Force delete, even if not merged (unmerged work is lost).
<code>git push origin --delete <branch></code>	Delete the branch on GitHub.

	Safe (fails if target exists / unmerged)	Force
Move (rename)	<code>-m</code>	<code>-M</code>
Copy	<code>-c</code>	<code>-C</code>
Delete	<code>-d</code>	<code>-D</code>

Rule: lowercase = safe, UPPERCASE = force. Renaming only changes your **local** branch. To rename on GitHub, push the new name (`git push -u origin <new>`), then delete the old remote branch.

5. Viewing history: `git log` and `git show`

`git log` vs `git show`

	<code>git log</code>	<code>git show</code>
Shows	many commits (history)	one thing, in detail
Default	commit messages, no code changes	commit message + full diff
Stops	at the beginning of history (shows the commit and all older ones)	after the object you asked for (shows only that one)

Example history (newest on top):

```
C4 Add student search  <- HEAD
C3 Add PHPStan config
C2 Add tests
C1 Initial commit
```

What each command prints:

```
git log C3  -> C3, C2, C1  (goes back to the
start)
git show C3  -> only C3 + its code changes
git log -1 -p C3  -> same as git show C3
```

Picture it this way: `git log` reads a diary from a given page back to page 1. `git show` opens one page and reads it closely.

`git log`: useful forms

Command	Use case
<code>git log --oneline</code>	Compact: short hash + message per line
<code>git log --graph --oneline --all</code>	Branch graph of every branch (the one to remember)
<code>git log -5</code>	Only the last 5 commits
<code>git log -p</code>	Each commit with its code changes
<code>git log --stat</code>	Which files changed in each commit, and how much
<code>git log -- <path></code>	Only commits that touched this file or folder
<code>git log --author="sachin"</code>	Commits by one person
<code>git log --since="2 weeks ago"</code>	Commits in a time range
<code>git log --grep="login"</code>	Commits whose message contains a word
<code>git log main..feature</code>	Commits on <code>feature</code> that are not yet in <code>main</code>

Reading the graph

```
* 9f3c2a1 (HEAD -> main1, origin/main1) Add student search
| * 7d4e8b2 (origin/main, main) Fix CI workflow
|/
* 3a1b5c9 Add PHPStan config
* 1e2f3d4 Initial commit
```

* = commit · | = branch line · / \ = split / merge · HEAD -> main1 = you are on `main1` · origin/main = GitHub's main at the last fetch. Read from top (newest) to bottom (oldest). Press **q** to leave the pager.

`git show`: useful forms

Command	Use case
---------	----------

<code>git show</code>	Latest commit: message + diff
<code>git show <hash></code>	A specific commit
<code>git show HEAD~1</code>	The commit before the latest (<code>HEAD~2</code> = two back)
<code>git show <branch></code>	Latest commit on that branch
<code>git show HEAD~2:path/to/file</code>	A file as it was two commits ago, without switching branches
<code>git show --stat</code>	Only the list of changed files
<code>git show --name-only</code>	Only file names

Why does the syntax say `<object>` ? Git stores commits, files (blobs), folders (trees) and tags as *objects*, and `git show` can display any of them.

6. Undoing things

Situation	Command	Notes
Discard edits in a file (not staged)	<code>git restore <file></code>	Edits are lost .
Unstage a file (keep the edits)	<code>git restore --staged <file></code>	Opposite of <code>git add</code> .
Fix the last commit's message, or add a forgotten file	<code>git commit --amend</code>	Only if not pushed yet.
Undo a pushed commit safely	<code>git revert <hash></code>	Adds a new commit that reverses it. History stays intact. Safe for shared branches.
Undo the last local commit, keep the changes	<code>git reset --soft HEAD~1</code>	Changes go back to staged.
Throw away the last local commit completely	<code>git reset --hard HEAD~1</code>	Destructive. Never on pushed commits.
Recover "lost" commits	<code>git reflog</code>	Shows everywhere HEAD has been, so you can find the hash and reset or switch back to it.

Golden rule: never rewrite history that others have pulled. For pushed commits, use `revert`. `reset --hard` and `push --force` are only for your own unshared work.

Stash: put work aside temporarily

Command	Use case
<code>git stash</code>	Save uncommitted changes and clean the folder (e.g. to switch branches quickly)
<code>git stash -u</code>	Also stash new (untracked) files
<code>git stash list</code>	See saved stashes
<code>git stash pop</code>	Bring back the latest stash and remove it from the list
<code>git stash drop</code>	Delete the latest stash

7. Combining branches

Command	What it does / when to use it
<code>git merge <branch></code>	Bring <code><branch></code> 's commits into the current branch (may create a merge commit).
<code>git rebase main</code>	Replay your branch's commits on top of the latest <code>main</code> , giving a straight history. Only on your own branch.
<code>git merge --abort</code>	Give up on a merge that has conflicts.
<code>git rebase --abort</code>	Give up on a rebase.

Resolving a conflict: open the file and find the markers <<<<<< / ===== / >>>>>>. Keep the correct code and delete the markers. Then run `git add <file>`, followed by `git commit` (for a merge) or `git rebase --continue` (for a rebase). VS Code shows "Accept Current / Incoming / Both" buttons for this.

8. Daily feature workflow (CI/CD style)

```
# 1. Start from an up-to-date main
git switch main
git pull

# 2. Create a branch for your work
git switch -c feature/student-search

# 3. Work, check, commit (repeat)
git status
git diff
git add .
git commit -m "Add student search endpoint"

# 4. Run the same checks as CI before pushing (from backend-php/)
composer ci

# 5. Push the branch (first time: -u)
git push -u origin feature/student-search

# 6. Open a Pull Request on GitHub, then wait for GitHub Actions to pass
# 7. Merge the PR into main, which triggers the deploy to Render

# 8. Clean up locally
git switch main
git pull
git branch -d feature/student-search
```

Why branches + PRs? Pushing a feature branch only runs the **checks**. Code reaches `main` (and gets deployed) only through a reviewed **merge**, so broken code is caught before production.

9. .gitignore: files Git should never track

```
# backend-php/.gitignore example
/vendor/           # rebuilt by composer install from composer.lock
.env              # secrets (Supabase keys!) never go into Git
.phpunit.cache/
info.php          # local test files
```

Command	Use case
<code>git rm --cached <file></code>	Stop tracking a file that was committed by mistake (keeps it on disk). Then add it to <code>.gitignore</code> .
<code>git check-ignore -v <file></code>	Explain which <code>.gitignore</code> rule ignores a file.

Commit `composer.lock`, not `vendor/`. A `vendor/` folder belongs to the system that created it: one installed in Docker (Linux) won't run on Windows. Always rebuild it with `composer install`.

10. Quick reference: the 15 commands you'll use most

Command	In one line	Command	In one line
---------	-------------	---------	-------------

<code>git status</code>	What changed?	<code>git switch -c x</code>	New branch + move to it
<code>git diff</code>	Show unstaged changes	<code>git switch x</code>	Go to branch
<code>git add .</code>	Stage everything	<code>git branch -vv</code>	Branches + upstreams
<code>git commit -m "..."</code>	Save a snapshot	<code>git merge x</code>	Bring x into current
<code>git pull</code>	Get + merge updates	<code>git stash / pop</code>	Park / restore work
<code>git push -u origin x</code>	First push of a branch	<code>git restore <f></code>	Discard file edits
<code>git log --graph --oneline --all</code>	Branch graph	<code>git revert <hash></code>	Safely undo pushed commit
<code>git show <hash></code>	One commit in detail		

Help any time: `git <command> -h` (short) · `git <command> --help` (full manual) · <https://git-scm.com/docs>